



TM481

Valve-based hydraulic drives

Prerequisites and requirements

Training modules	TM210 – The Basics of Automation Studio TM480 – The Basics of Hydraulics
Software	AS 4.1 or later
Hardware	---

Table of contents

1 Introduction.....	4
1.1 Learning objectives.....	4
2 The concept of valve-based hydraulic drive control.....	5
2.1 Control tasks.....	5
2.2 Mechatronics libraries for valve-based hydraulics.....	6
2.3 MTHydValve - Function blocks.....	6
3 Simulation model.....	8
3.1 Internal structure of the simulation model.....	8
3.2 Exercise: Simulation model.....	9
4 Simple positioning.....	11
4.1 The control loop.....	11
4.2 MTProfilePositionGenerator.....	11
4.3 MTHydValveLinearization.....	12
4.4 MTHydValvePositionController.....	14
4.5 Commissioning the closed control loop.....	15
5 Positioning with servo correction (optional).....	17
5.1 Servo correction.....	17
6 Technology Solution Hydraulic Valve Control.....	23
6.1 Technology Solution exercise.....	23
7 Force control.....	24
7.1 Control loop.....	24
7.2 MTHydValveForceController.....	24
7.3 Servo correction.....	25
8 Alternating positioning and force control.....	28
8.1 The principle of alternating control.....	28
9 Project development.....	31
9.1 Helpful information regarding project preparation.....	31
9.2 Commissioning guidelines.....	34

1 Introduction

The TM481 training module deals with the topic of controlling hydraulic cylinders by actuating hydraulic valves. It will also present sample solutions programmed using the MTHydValve library. To improve comprehension, each control loop will be implemented and analyzed directly in Automation Studio.

1.1 Learning objectives

This training module uses selected examples and exercises to help you learn about how to implement control loops with valve-based hydraulic drives.

- Participants will learn about the structure of library MTHydValve.
- Participants will learn how to use the function blocks in library MTHydValve.
- Participants will learn the function blocks' most important parameters and what they do.
- Participants will learn the most important machine parameters needed to configure the function blocks.
- Participants will learn how to develop and commission positioning tasks for hydraulic axes.
- Participants will learn about alternating position and force control applications.

2 The concept of valve-based hydraulic drive control

The following section will present a closed loop control concept for hydraulic drives. First, the hardware configuration will be dealt with. Subsequently, the closed loop control strategies, which are carried out in the Automation Studio library MTHydValve, are developed step by step.

B&R's approach to controlling hydraulic drives is built on products from their standard product range rather than specialized modules. Control algorithms are executed directly on the CPU. A wide range of CPUs (e.g. Power Panel, X20 CPU, APC) allows users to get the exact level of performance they need, as determined by the number of hydraulic axes and the desired cycle and response times. The following image shows a sample configuration with an X20 system as the CPU.

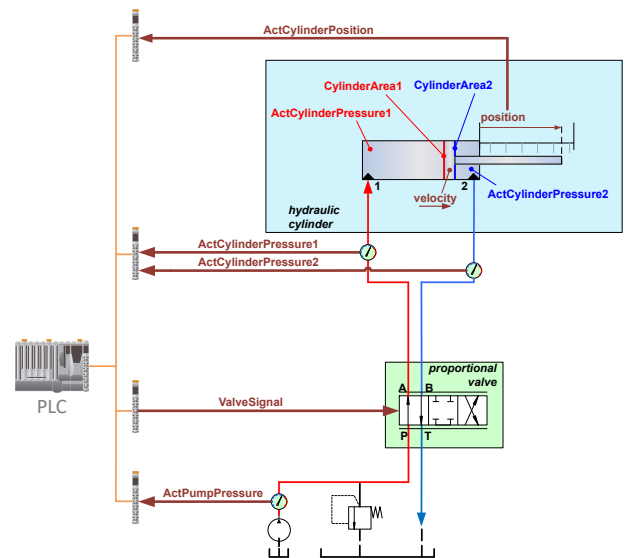


Figure 1: Example of a hardware concept for valve-based hydraulics

Sensors for position, pressure, force and flow rate are integrated using appropriate input modules (in this example, the current position is read using an X20 encoder module). Manipulated variables for valve positions are transferred to the valves via analog output modules that, like the input modules, are also connected via X2X.

Valve-based hydraulic control is computed as software on the controller. As a result, the software control loop can be put together easily and flexibly using a building block system. The image to the right depicts a control loop for positioning that uses a profile generator, a controller and valve linearization. The profile generator is a function block of the library MTPProfile, the controller and valve linearization are components of the library MTHydValve. In this example, the function blocks are executed on the X20 CPU.

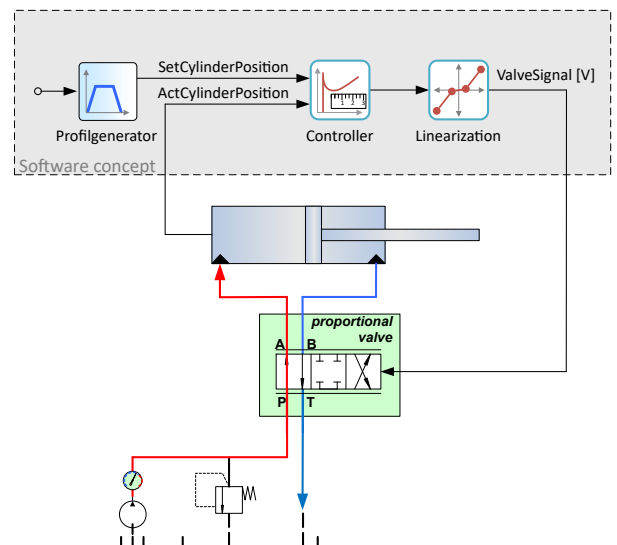


Figure 2: Example of a software concept for valve-based hydraulics

2.1 Control tasks

Valve-based hydraulic control tasks generally involve the following elements:

- Position control with or without a profile generator
- Speed control
- Force/Pressure control

2.2 Mechatronics libraries for valve-based hydraulics

The mechatronics libraries offer the right function blocks to help create optimal solutions for the valve-based hydraulics applications listed above. General functions such as signal filtering (MTFilter) and profile generation (MTProfile) are covered by libraries from the "Basic Controller Design" package. The MTHydValve library is for hydraulic-specific functions for valve-based applications. Additionally, there is the library MTHydPump and MTHydGen. The MTHydPump library contains functionalities for open and closed control loops of hydraulic servo pump drives. The MTHydGen library contains additional general hydraulic functions such as adaptive position setpoint correction. In this training module, the libraries from the "Basic Controller Design" package are used in association with the MTHydValve library.

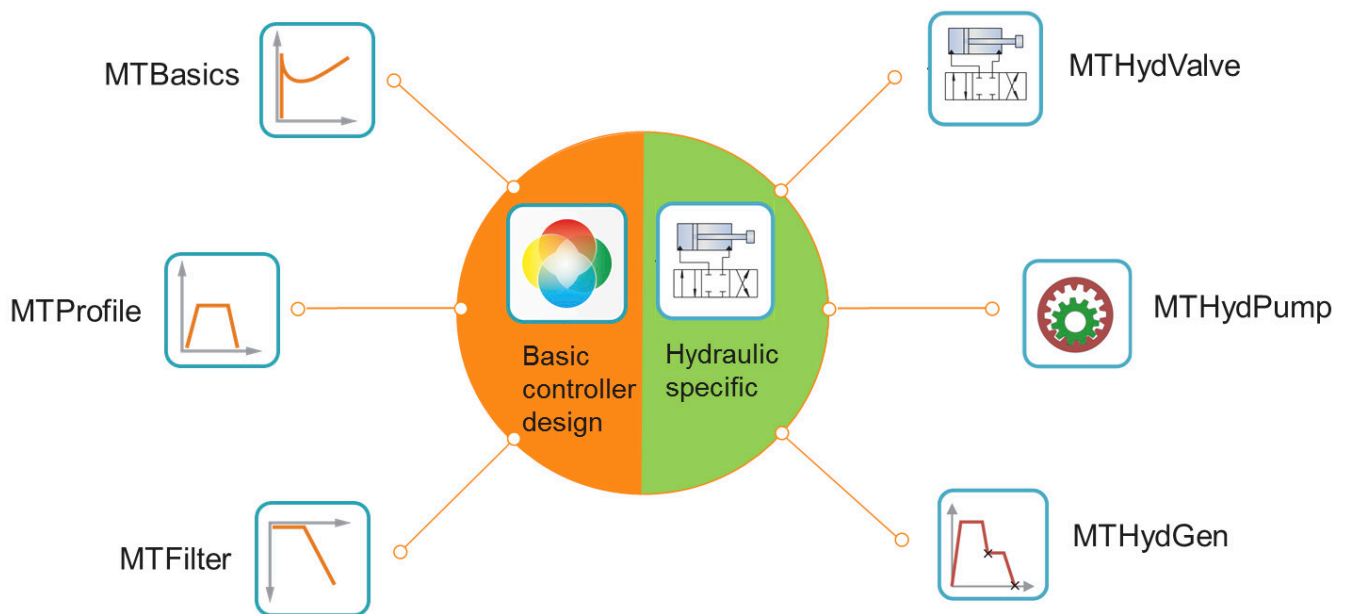


Figure 3: Mechatronics libraries

2.3 MTHydValve - Function blocks

The following image shows all function blocks that the library MTHydValve contains.

	Position controller PID Feedforward + Servo correction
	Force controller PID Feedforward + Servo correction
	Valve linearization Static valve linearization
	Simulation model Physics of a hydraulic axis

Figure 4: Function blocks in library MTHydValve

3 Simulation model

The function block MTHydValveSimulationModel is used to simulate a valve-based hydraulic drive with a hydraulic cylinder. The cylinder parameters, valve parameters and general hydraulic parameters can be configured at will.

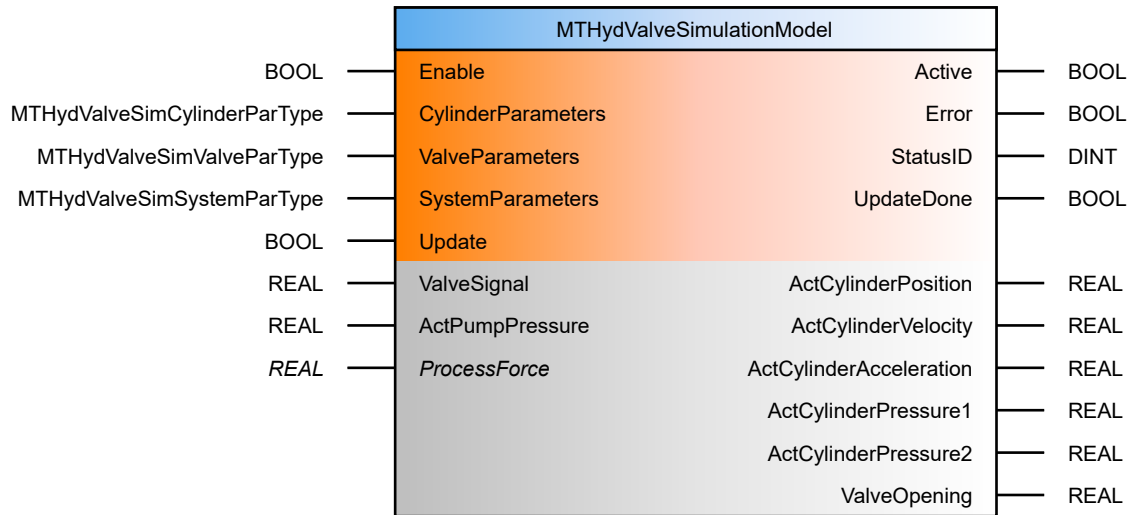


Figure 5: Function block MTHydValveSimulationModel



Programming \ Libraries \ Mechatronic libraries \ Hydraulics \ MTHydValve \ Function blocks \ MTHydValveSimulationModel

3.1 Internal structure of the simulation model

The pump pressure and valve signal are required as a cyclical input. The valve signal must be specified between -1 and +1.

The cylinder chamber pressures, cylinder position, cylinder speed as well as cylinder acceleration are calculated cyclically. In addition, the current valve opening is output as information.

Optionally, a process force that is taken into account when simulating system variables can be specified.

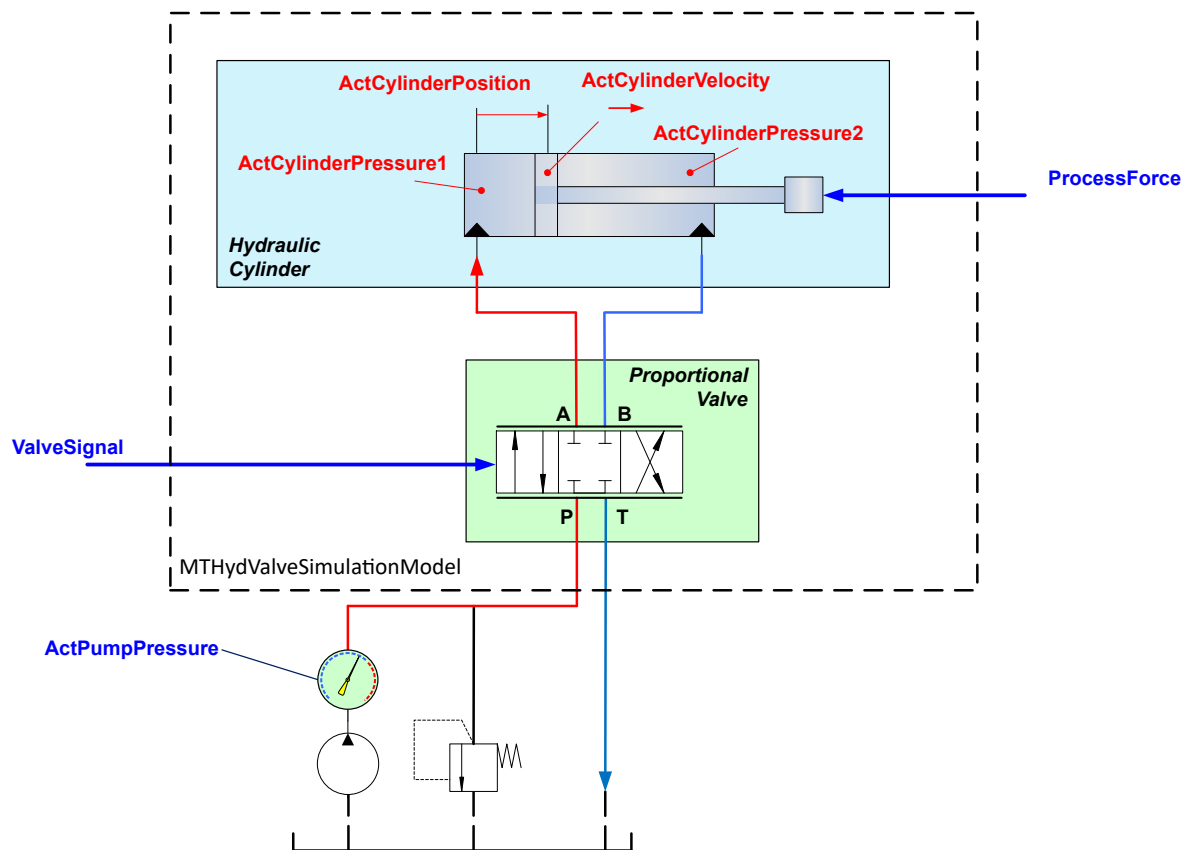


Figure 6: Internal structure MTHydValveSimulationModel

3.2 Exercise: Simulation model

Exercise: Implement simulation model in AS

The function block MTHydValveSimulationModel must be implemented in a new Automation Studio program with the name SimulationModel. Standard values for the cylinder, valve and system parameters are already specified. The following valve parameters with the associated valve characteristic curve should be configured as part of an exercise.

Valve parameters

- ValveNominalVolumeFlowPA = 100.0 [l/min]
- ValveNominalVolumeFlowPB = 100.0 [l/min]
- ValveNominalPressureDrop = 5.0 [bar]
- ValveLeakageFactor = 0.001 [l/min/bar]
- NumNodesValveCurve = 5

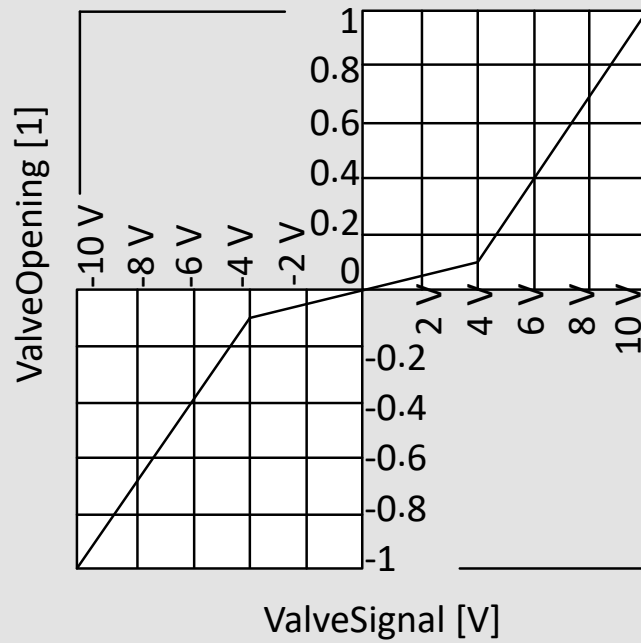


Figure 7: Valve characteristic curve

4 Simple positioning

This section provides information about the simple positioning of hydraulic cylinders. The first thing covered is the control loop itself. We will then take a look at the individual components of a control loop (function blocks). An exercise is prepared for every function block. The objective, according to these exercises, is to have the entire closed control loop for the hydraulic positioning implemented in Automation Studio.

4.1 The control loop

In most cases, motion profiles are required for hydraulics which are subject to predefined limitations with regard to maximum acceleration and/or jolt as well as maximum speed. In this case, the library MTPProfile provides the MTPProfilePositionGenerator block to create a motion profile with the required corresponding characteristics. It is also possible to use other profile generators, for example from an electrical axis. The setpoints from the motion profile are transferred to controller MTHydValvePositionController. The controller output is then readied for the valve using the MTHydValveLinearization function block (compensation of the static valve characteristic curve).

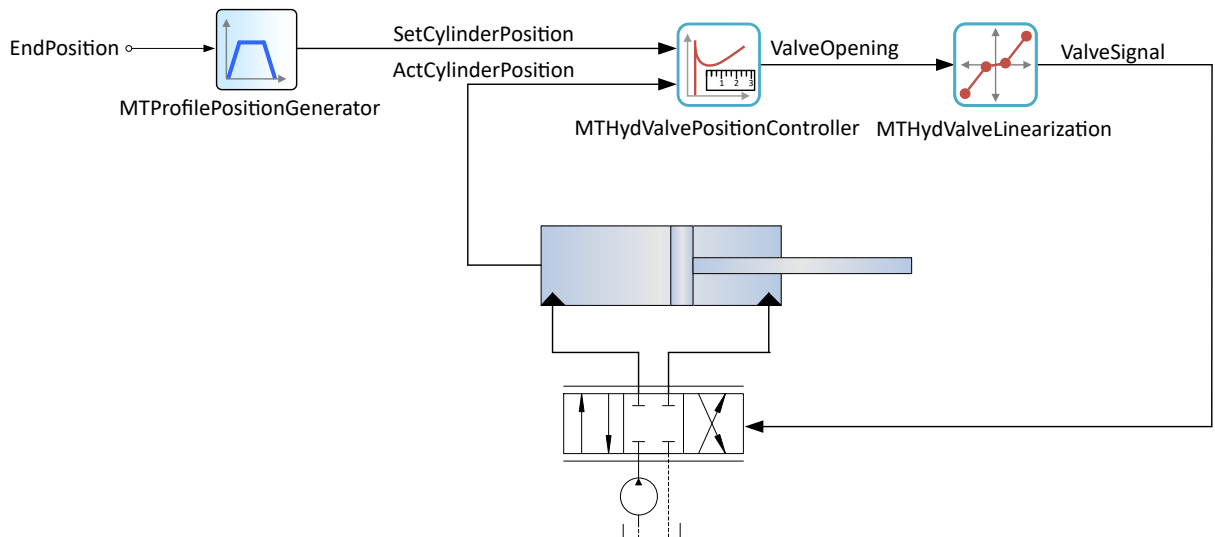


Figure 8: Control loop of a simple positioning

4.2 MTPProfilePositionGenerator

The MTPProfilePositionGenerator function block allows you to generate a motion profile (starting from the current position) which can be defined by specifying the target position and a maximum speed, acceleration and deceleration.

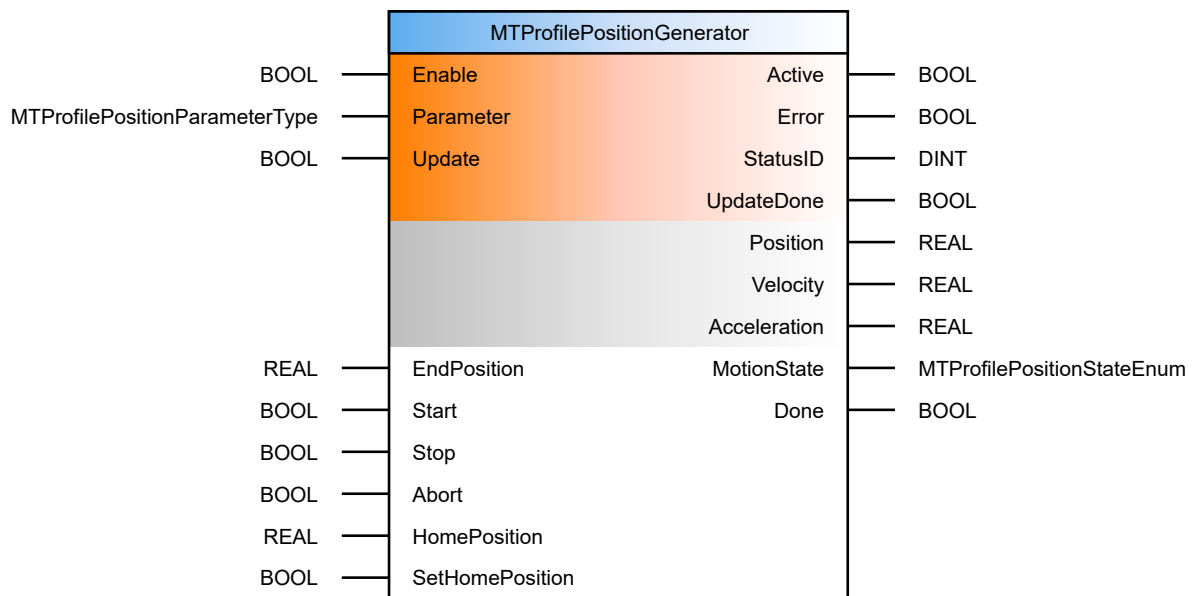


Figure 9: Function block MTPProfilePositionGenerator



Programming \ Libraries \ Mechatronic libraries \ Basic Controller Design \ MTPProfile \ Function blocks \ MTPProfilePositionGenerator



Keep in mind that control is not yet active since the movement profile created by the movement profile generator represents a position setpoint curve. Nevertheless, take the characteristics of the hydraulic drive into consideration when configuring this setpoint profile.

This type of cyclic movement profile occurs in almost all machine-related tasks since recurring tasks are most often handled by the drive.

4.2.1 Profile generator exercise

Exercise: Implement profile generator in AS

The function block MTPProfilePositionGenerator must be implemented in a new Automation Studio program with the name ProfileGenerator. The following profile parameters should be configured:

Profile parameters

- VelocityPosDirection = 150.0 [mm/s]
- VelocityNegDirection = 150.0 [mm/s]
- AccelerationPosDirection = 2000.0 [mm/s²]
- AccelerationNegDirection = 2000.0 [mm/s²]
- DecelerationPosDirection = 2000.0 [mm/s²]
- DecelerationNegDirection = 2000.0 [mm/s²]
- JoltTime = 0.2 [s]

The generated trajectory must be checked in the Trace window to see if it corresponds to the specified values.

4.3 MTHydValveLinearization

The function block MTHydValveLinearization allows the static linearization of valves. This can be used to compensate static valve characteristic curves, which are usually nonlinear. The function block MTHydValveLinearization converts a given valve opening into the required valve signal (e.g. V, mV, digits, etc.).

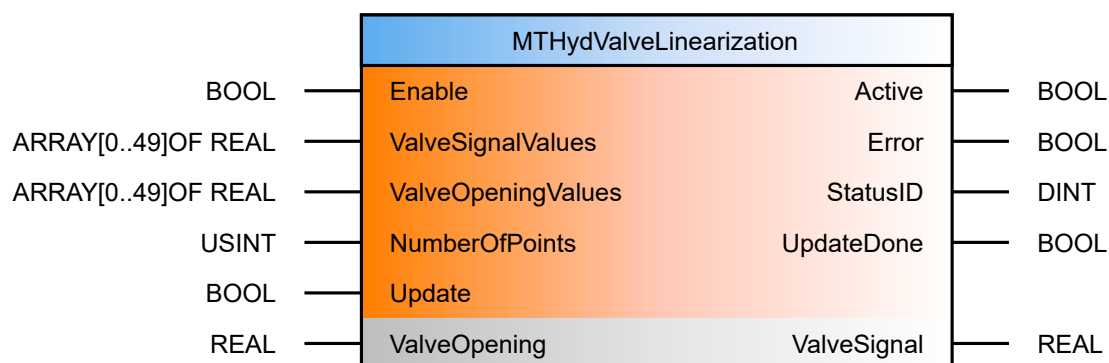


Figure 10: Function block MTHydValveLinearization



Programming \ Libraries \ Mechatronic libraries \ Hydraulics \ MTHydValve \ Function blocks \ MTHyd-ValveLinearization

For this conversion, the static characteristic curve of the valve is needed. This is specified in the form of coordinates. Areas between the coordinates are calculated using linear interpolation. The inverted valve characteristic curve is calculated in the function block when it is enabled and at every update. The inverted valve characteristic curve is used by the function block MTHydValveLinearization to calculate a valve signal for the desired valve opening. The valve signal unit is defined with the configuration of the nodes (e.g. V, mV, digits, etc.).

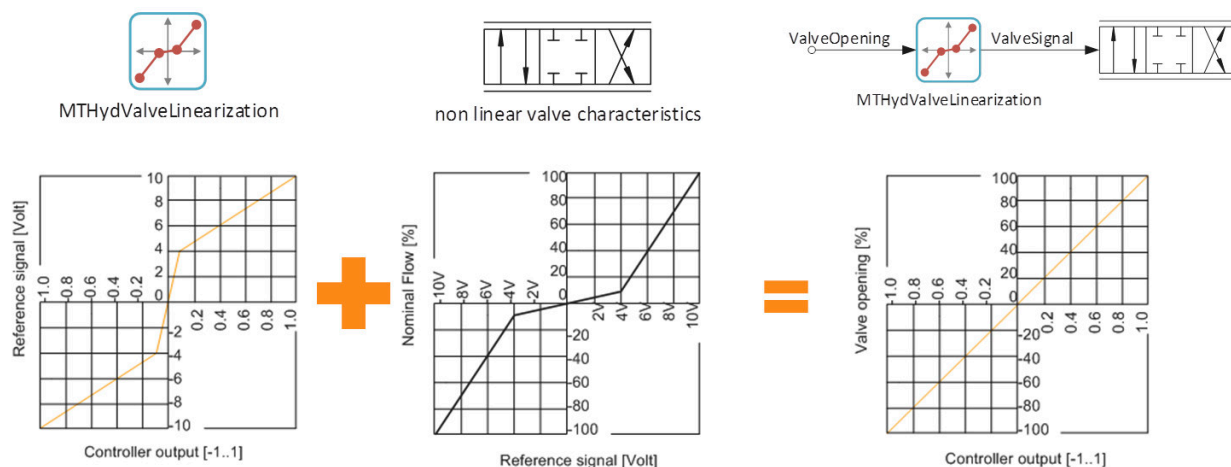
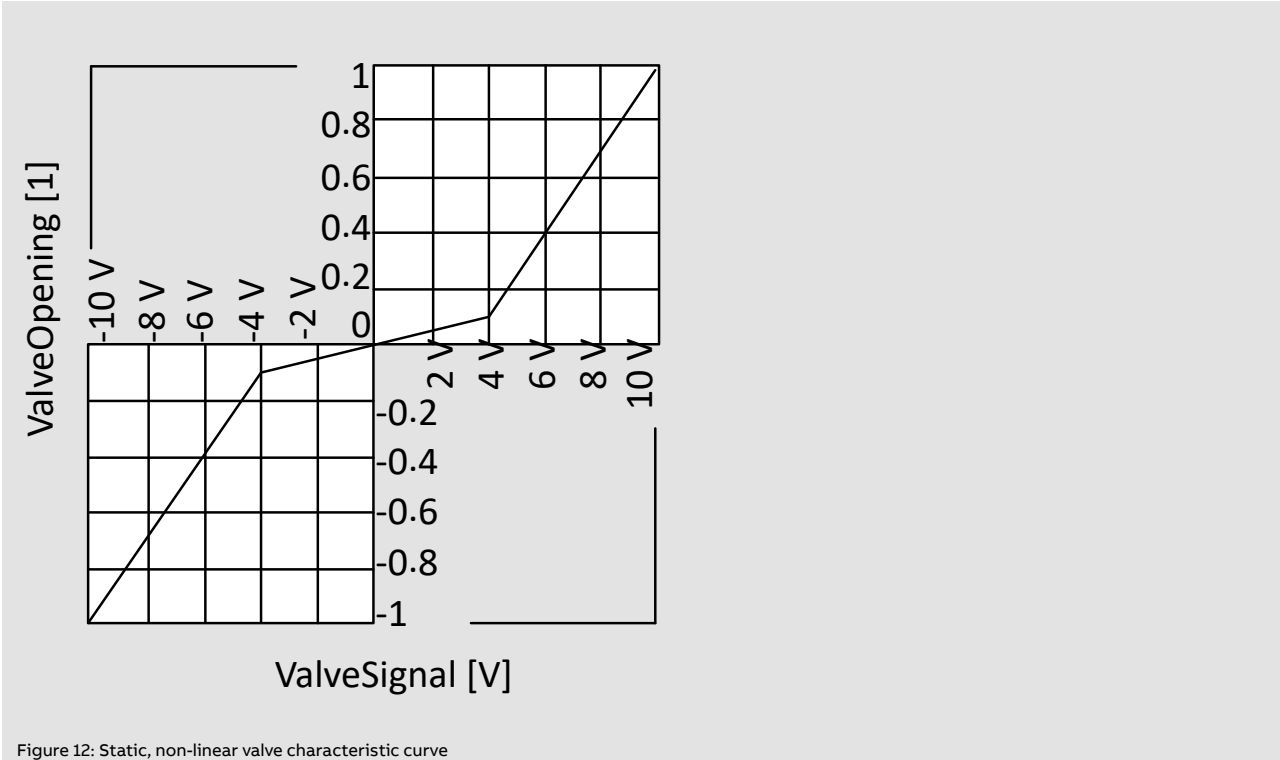


Figure 11: Compensation of the static, non-linear valve characteristic curve

4.3.1 Valve linearization exercise

Exercise: Implement valve linearization in Automation Studio

The function block MTHydValveLinearization must be implemented in a new Automation Studio program with the name ValveLinearization. The following valve characteristic curve must be configured:



4.4 MTHydValvePositionController

The MTHydValvePositionController function block is a controller optimized for valve-based hydraulics. It can be used for flexible positioning with the aid of a profile generator such as the function block MTProfilePositionGenerator.

The function block MTHydValvePositionController includes two components, a PID controller and servo correction. More information about using servo correction can be found in the next section.

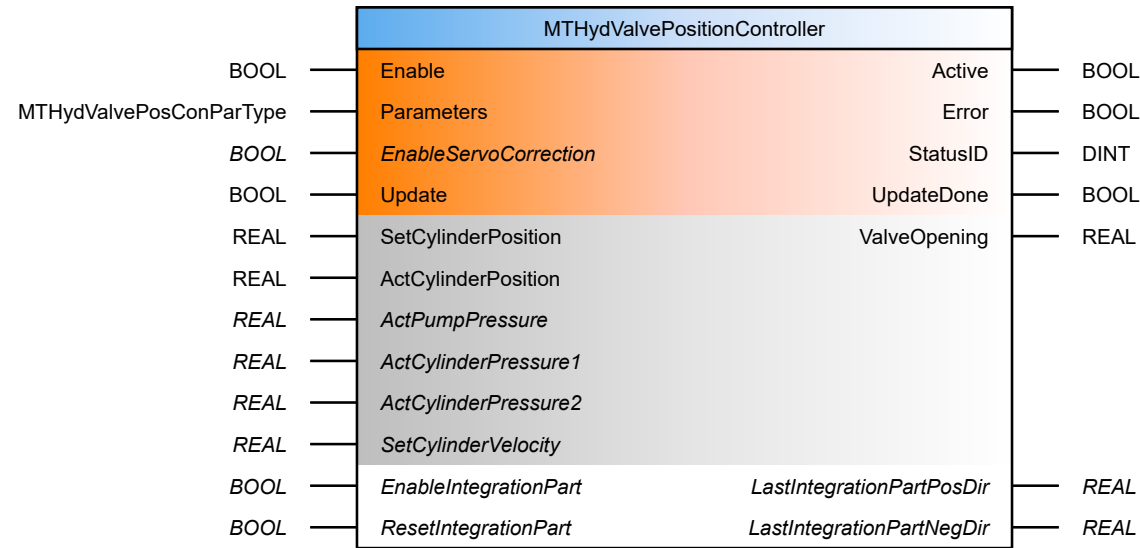


Figure 13: Function block MTHydValvePositionController

Programming \ Libraries \ Mechatronic libraries \ Hydraulics \ MTHydValve \ Function blocks \ MTHyd-ValvePositionController

In the simplest case, the function block MTHydValvePositionController is operated as a pure PID controller (EnableServoCorrection = FALSE). In this mode, only the PID parameters (Parameters.PID) have to be configured. SetCylinderPosition and ActCylinderPosition are the only cyclical inputs needed for this. The manipulated variable is connected to the ValveOpening output. This variable's default range is {-1..+1}.



Changes to the parameter structure and the input EnableServoCorrection do not take effect until the function block is enabled or there is a rising edge of the update input.



If servo correction is additionally used, the cyclical inputs ActPumpPressure, ActCylinderPressure1, ActCylinderPressure2, SetCylinderVelocity, and the parameter structure Parameters.MachineData must be configured accordingly.

4.4.1 Position controller exercise

Exercise: Implement the position controller in Automation Studio

The MTHydValvePositionController function block must be implemented in a new Automation Studio program with the name Control. In the first step, the position controller must be configured just as a P controller. The position setpoint and actual position must be changed manually for now and the output ValveOpening observed by using different proportional gains. This still does not correspond to any closed control loop. If the manual tests work, the control loop must be concluded with the next exercise.

4.5 Commissioning the closed control loop.

If the previous exercises of the training module have been worked through correctly, there should now be four programs independent of each other in Automation Studio.

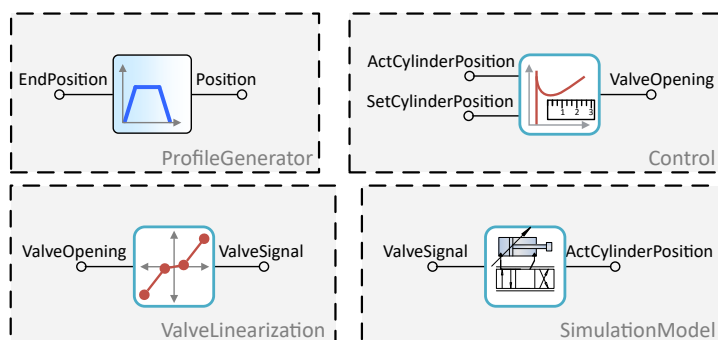


Figure 14: Programs in Automation Studio



The programs ProfileGenerator, Control, ValveLinearization and SimulationModel must be connected together via global variables and accordingly to the control system.

4.5.1 Closed control loop exercise

Exercise: Include control loop in Automation Studio

For the closed control loop, the function blocks must communicate with each other in the programs ProfileGenerator, Control, ValveLinearization and SimulationModel. This should be implemented with global variables.

Procedure

- Set up the following global variables:
gEndPosition, gStart, gSetPosition, gValveOpening, gValveSignal, gActCylinderPosition
- Connect global variables with the inputs and outputs of the function blocks.

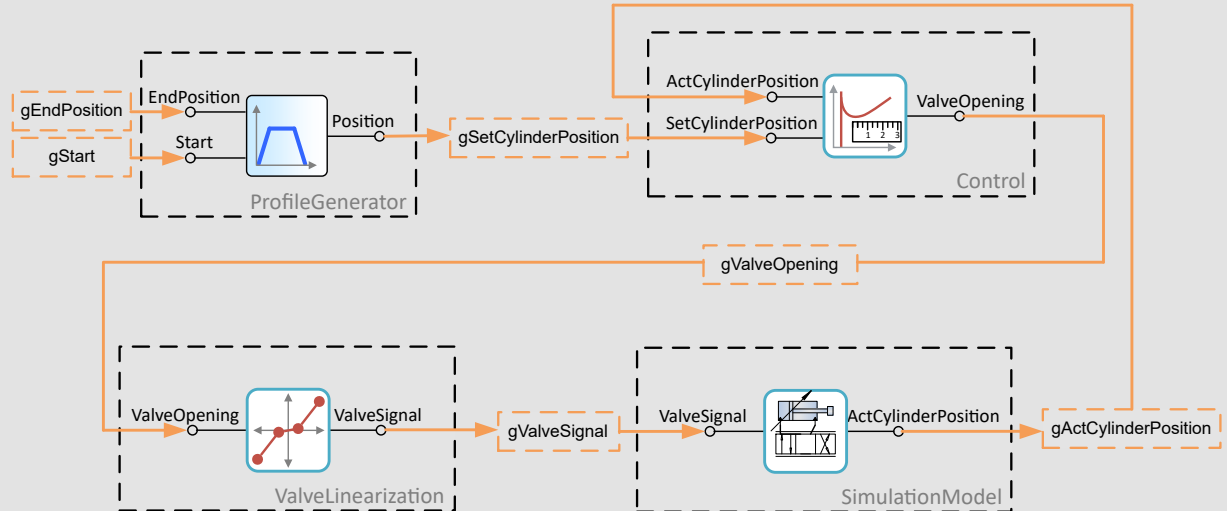


Figure 15: Programs in Automation Studio connected via global variables

- Test the closed control loop.
 - Configure position controller as a P controller (start with gain = 0.01).
 - Specify a position in the Watch window for `gEndPosition` and start the positioning with `gStart`.
 - If the function blocks have been correctly connected together, closed-loop control for the end position should now be started.
 - Checking the closed-loop control using the Trace (`ActCylinderPosition`, `SetCylinderPosition`, `ActCylinderVelocity`, `ValveOpening`)

5 Positioning with servo correction (optional)

In this case, servo correction is added to the simple positioning control loop. The goal is to improve the precision and dynamics of the positioning task using servo correction.

5.1 Servo correction

Up until now, we have assumed that the volumetric flow rate (and with it, the cylinder speed) has been roughly proportional to the valve opening. As explained in the TM480 training module, however, there is a direct relationship between the drop of pressure at the valve and the volumetric flow rate. The approach described below takes this relationship into consideration in order to improve positioning.

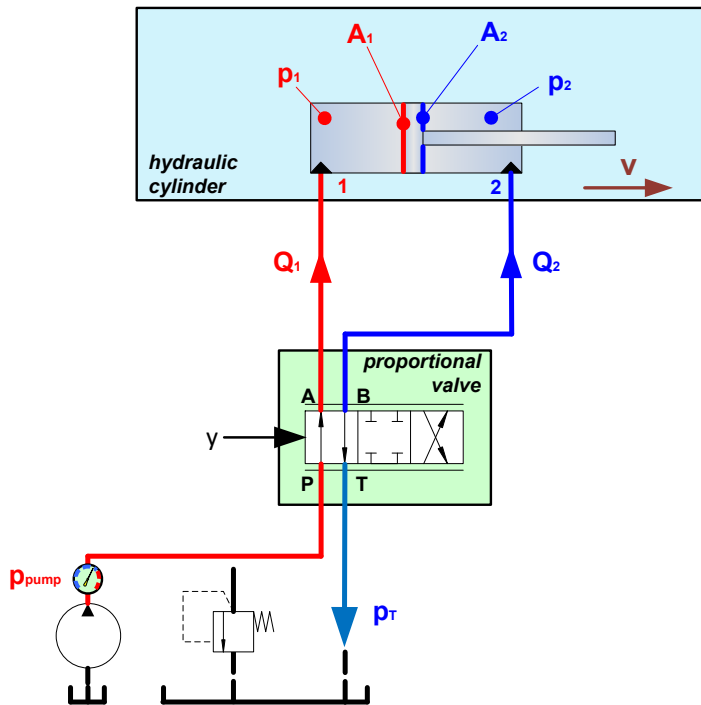


Figure 16: Hydraulic circuit with physical quantities

$$Q_1 = y * Q_{nomA} * \sqrt{\frac{p_{pump} - p_1}{\Delta p_{nom}}}$$

$$Q_2 = -y * Q_{nomB} * \sqrt{\frac{p_2 - p_T}{\Delta p_{nom}}}$$

In addition, the necessary volumetric flow rate can be defined for a certain velocity setpoint (using the cylinder surface areas A_1 and A_2).

$$Q_1 = v * A_1 \quad Q_2 = -v * A_2$$

By applying servo correction, adjusting for the cylinder surface areas and performing valve linearization, the cylinder's velocity setpoint is converted into a corresponding valve voltage that compensates for the loss of pressure at the valve, the difference in surface areas and the valve characteristic curve.

$$v * A_1 = y * Q_{nomA} * \sqrt{\frac{p_{pump} - p_1}{\Delta p_{nom}}}$$

$$y = v * \frac{A_1}{Q_{nomA}} * \sqrt{\frac{\Delta p_{nom}}{p_{pump} - p_1}}$$

$$y = f(v)$$

Physical quantity in the hydraulic circuit

- p_{pump}Current pump pressure (ActPumpPressure)
- p_TCurrent pressure in the tank
- yValve opening (ValveOpening unit: [1])
- Q_1Current volumetric flow rate into cylinder chamber 1
- Q_2Current volumetric flow rate into cylinder chamber 2
- p_1Current pressure in cylinder chamber 1 (ActCylinderPressure1)
- p_2Current pressure in cylinder chamber 1 (ActCylinderPressure2)
- A_1Cylinder area in cylinder chamber 1 (CylinderArea1)
- A_2Cylinder area in cylinder chamber 2 (CylinderArea2)
- vCylinder speed
- Q_{nomA}Nominal valve volumetric flow rate at valve connection A (ValveNomVolumeFlowPA)
- Q_{nomB}Nominal valve volumetric flow rate at valve connection B (ValveNomVolumeFlowPB)
- Δp_{nom}Nominal pressure drop at valve edge (ValveNomPressureDrop)

In short, servo correction converts cylinder velocities into valve openings. This makes it additionally possible to use the velocity setpoint of the profile generator. This velocity setpoint is passed directly to the servo correction as a feed-forward value after the position controller.

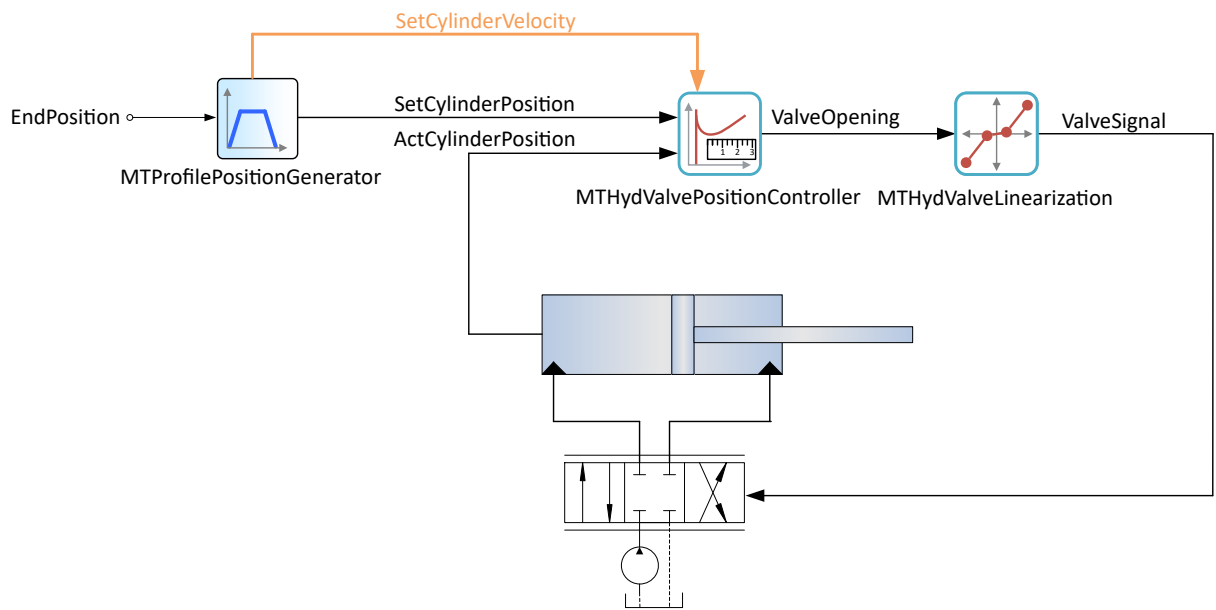


Figure 17: Cylinder velocity setpoint as feed-forward control

The following extended positioning block diagram shows that the PID controller now outputs a velocity setpoint. Servo correction dynamically converts this velocity setpoint into a valve opening. Applying valve linearization to the valve opening then results in the valve voltage.

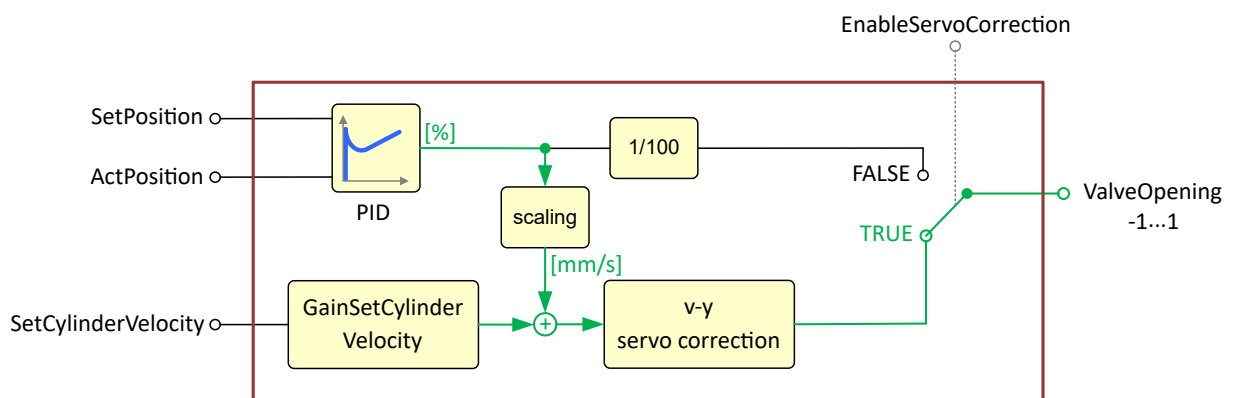


Figure 18: Internal structure of the MTHydValvePositionController

In some hydraulic applications, the system pressure is not always constant at all operating points. A change in system pressure over a large range (e.g. more than 10 % of the nominal pressure) can have negative effects on the control behavior. In this case, it would be better to measure the system pressure with a sensor and use the actual measurement rather than a constant.

The loss of pressure at the valve is either measured directly using pressure sensors and calculated accordingly or calculated indirectly using the dynamic process force.



In order to use the servo correction of the function block MTHydValvePositionController, the cyclical inputs ActPumpPressure, ActCylinderPressure1, ActCylinderPressure2, SetCylinderVelocity and the parameter structure Parameters.MachineData just have to be configured accordingly. Servo correction is enabled with the input EnableServoCorrection.

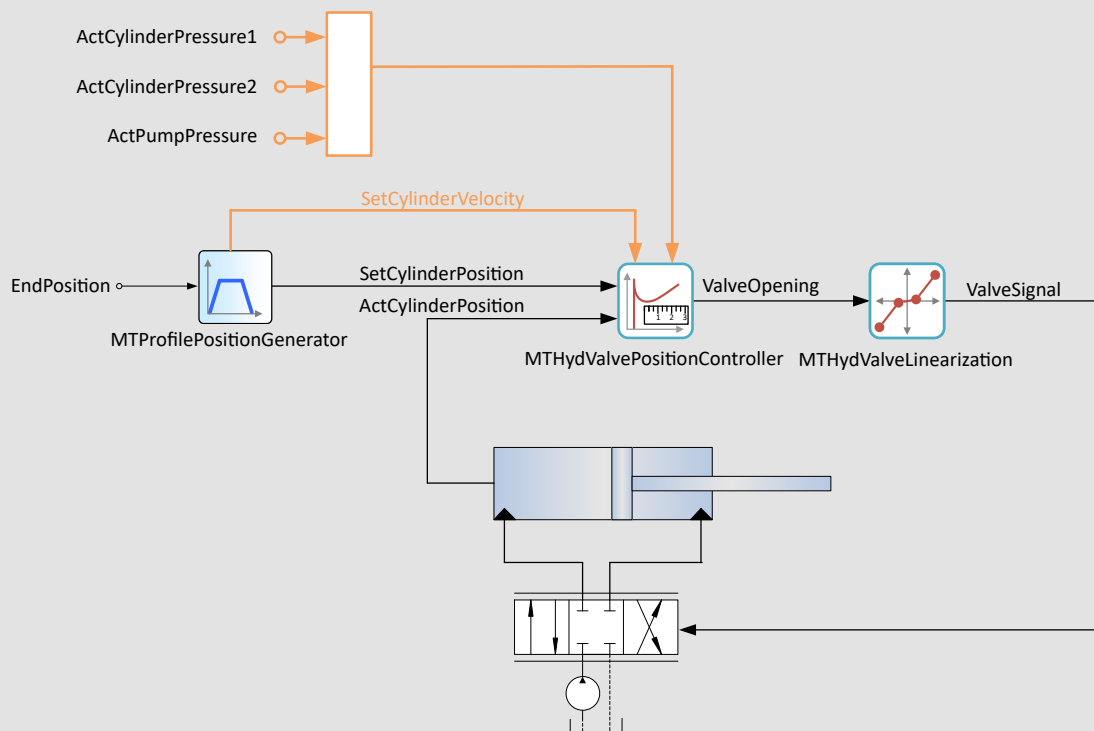


Figure 19: Cyclical inputs for the servo correction of the position controller



As can be seen from the previous image "Internal structure of the MTHydValvePositionController", the PID output is scaled to the velocity setpoint when enabling the servo correction. This way, the gain of the PID controller has a different effect than when operating without servo correction.

5.1.1 Position control with servo correction exercise

Exercise: Execute position control with servo correction

The closed control loop already available in Automation Studio must now, in addition, use servo correction for the positioning. Now additional outputs of the simulation model and the profile generator must be cyclically transferred to the position controller. For this, global variables are used again.

Procedure

- Set up the following global variables:
gActPumpPressure, gSetCylinderVelocity, gActCylinderPressure1, gActCylinderPressure2
- Set the variable gActPumpPressure in the program Control to 200 bar: gActPumpPressure = 200
- Connect global variables with the inputs and outputs of the function blocks.

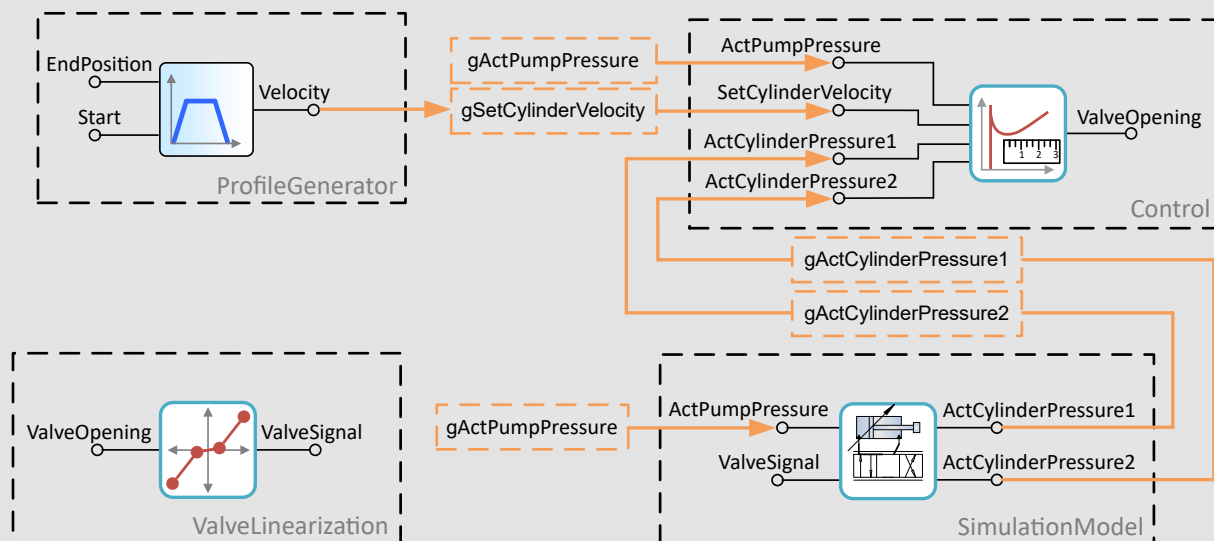


Figure 20: Cyclical variables for servo correction connected via global variables

- Configure machine data for the servo correction of the position controller:
 - CylinderArea1 = 165.0
 - CylinderArea2 = 86.0
 - MaxPumpPressure = 300.0
 - ValveNominalVolumeFlowPA = 100.0
 - ValveNominalVolumeFlowPB = 100.0
 - ValveNominalPressureDrop = 5.0
 - CoordinateSystem = mtHYDVALVE_CYLCHAMBER1_POSDIR
 - ValveCylinderConnection = mtHYDVALVE_PORT_A_TO_CYLCHAMBER1
- Enable servo correction: EnableServoCorrection = TRUE
- Execute and test positioning with servo correction
 - Specify a position in the Watch window for gEndPosition and start the positioning with gStart.
 - Checking the closed-loop control using the Trace (ActCylinderPosition, SetCylinderPosition, ActCylinderVelocity, ActCylinderPressure1, ActCylinderPressure2, ValveOpening).
 - Positioning only with servo correction (Gain = 0 !)
 - Positioning only with PID (EnableServoCorrection = FALSE)
 - Positioning with PID and with ServoCorrection (Gain > 0, EnableServoCorrection = TRUE)
 - Comparison of the above-mentioned control methods.

5.1.2 Servo correction without pressure sensors

If there are no pressure sensors to measure the pressure of the two cylinder chambers, then the servo correction is calculated using the friction force and process force. The process force can be measured using a force sensor on the piston rod, for example, or derived from the gravitational force. The following rule of thumb can be used to calculate the friction:

Approx. 10 N friction force per mm rod diameter.

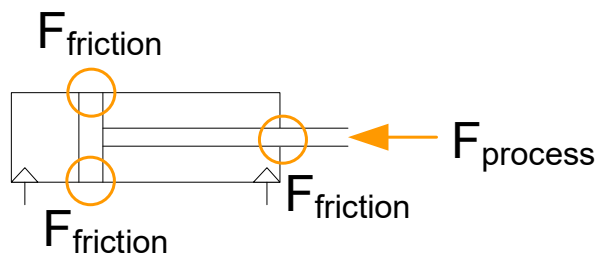


Figure 21: Friction in the cylinder

If the chamber pressure values are not available, function block MTHydValvePressureObserver can be used.

5.1.2.1 MTHydValvePressureObserver

Function block MTHydValvePressureObserver calculates the pressures in the cylinder chambers if the measurements are not available.

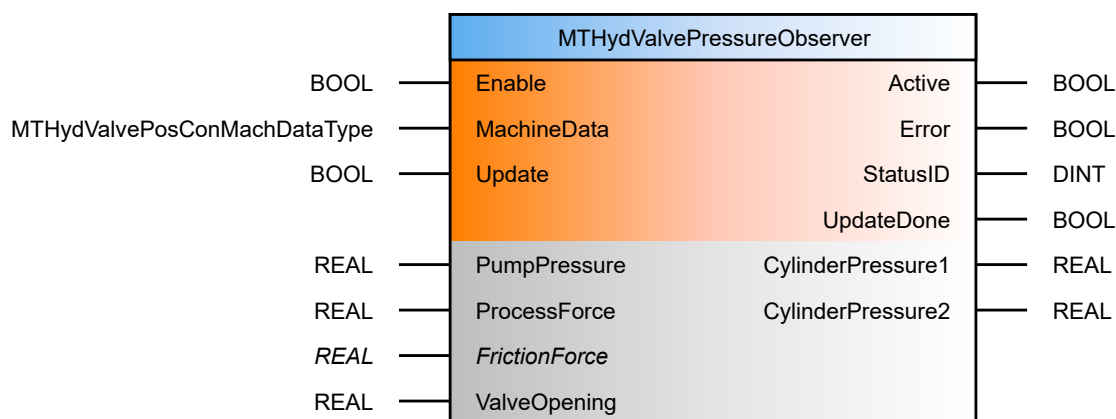


Figure 22: Function block MTHydValvePressureObserver



Programming \ Libraries \ Mechatronic libraries \ Hydraulics \ MTHydValve \ Function blocks \ MTHydValvePressureObserver

If function block MTHydValvePressureObserver is used within a hydraulic control loop, it is only necessary to connect the two outputs CylinderPressure1 and CylinderPressure2 with the controller. The following figure shows the control concept for position control without measured cylinder chamber pressures.

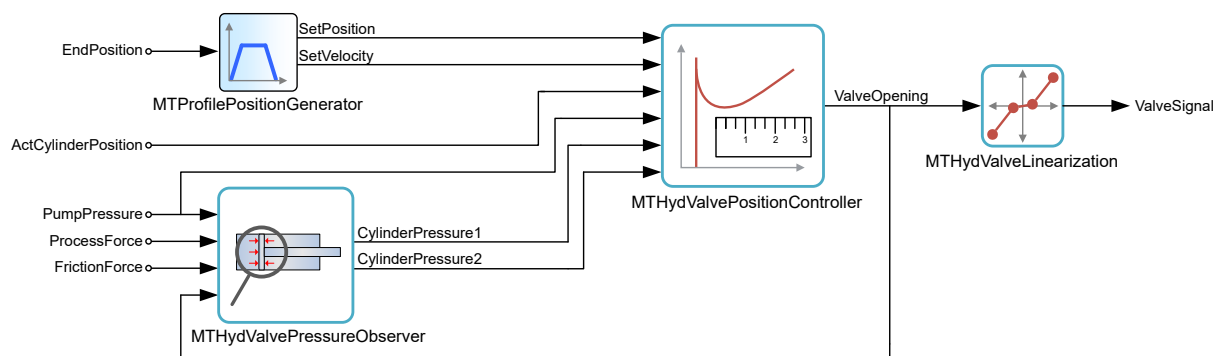


Figure 23: Position control without measured cylinder chamber pressures

6 Technology Solution Hydraulic Valve Control

The Technology Solution "Hydraulic Valve Control" contains the most important functions for operating a valve-based hydraulic drive. With simulation, these functions can be tested without needing a real machine. Just a few changes are then needed to very quickly perform commissioning and get a hydraulic axis moving.

The Technology Solution is a completely prepared package with the following content:

- Fully programmed position and force control as "open source" programs
- Visualization
- Simulation functionality
- Own configuration
- Help

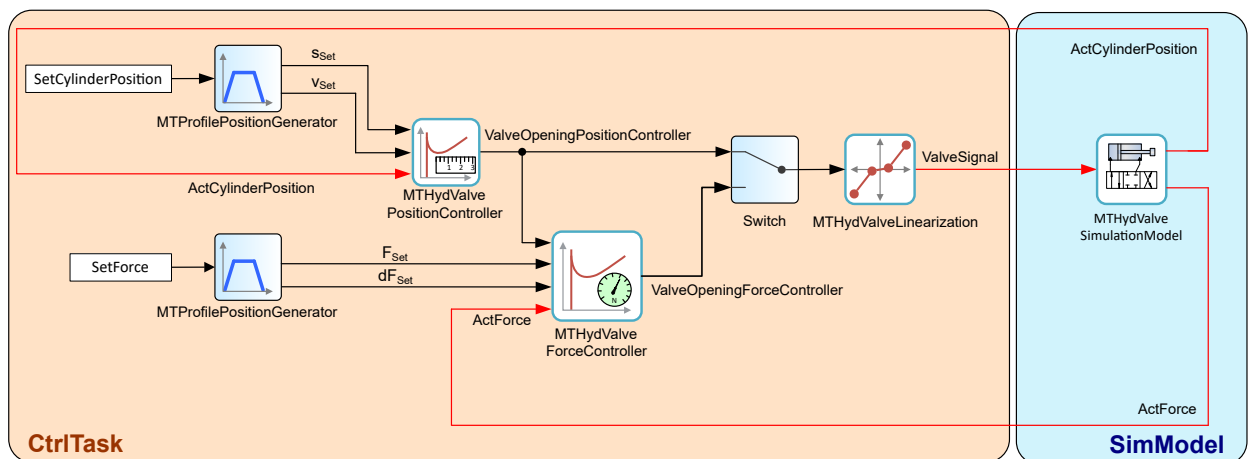


Figure 24: Block diagram of the Technology Solution Hydraulic Valve Control



Mechatronics \ Hydraulics \ Hydraulic Valve Control

6.1 Technology Solution exercise

Exercise: Position control with Technology Solution

The Technology Solution Hydraulic Valve Control must be put into operation and the HMI application can be used to execute a position control.

Procedure

- Install the most current version of the Technology Solution "Hydraulic Valve Control" via upgrade.
- Add the Technology Solution in Automation Studio
- Enable the configuration "HydraulicValveControl" added by the Technology Solution.
- Update Runtime version.
- Transfer to target system.
- Connect with VNC viewer.
- Execute positioning with HMI application.
- Observe in Trend.

7 Force control

This section provides information about force control with hydraulic cylinders. The first thing covered is the control loop itself. Then force control will be discussed. As with the position controller, there is also servo correction with force control. As an exercise, force control with and without servo correction is carried out using the Technology Solution "Hydraulic Valve Control".

7.1 Control loop

The closed control loop for force control looks similar to the control loop for positioning. In most cases, a profile generator is also used here, which now generates the force setpoint. Here, for example, the function block MTPProfilePositionGenerator from the MTPProfile library can also be used again. The output Position would, however, now be used as SetForce [N], and the output Velocity would correspond to the rate of force increase SetForceRate [N/s]. Alternatively, other force profile generators can also be used. This force profile is processed in the force controller MTHydValveForceController. Subsequently, a valve opening is calculated again and sent on to the valve linearization. This in turn calculates a valve signal for the proportional valve.

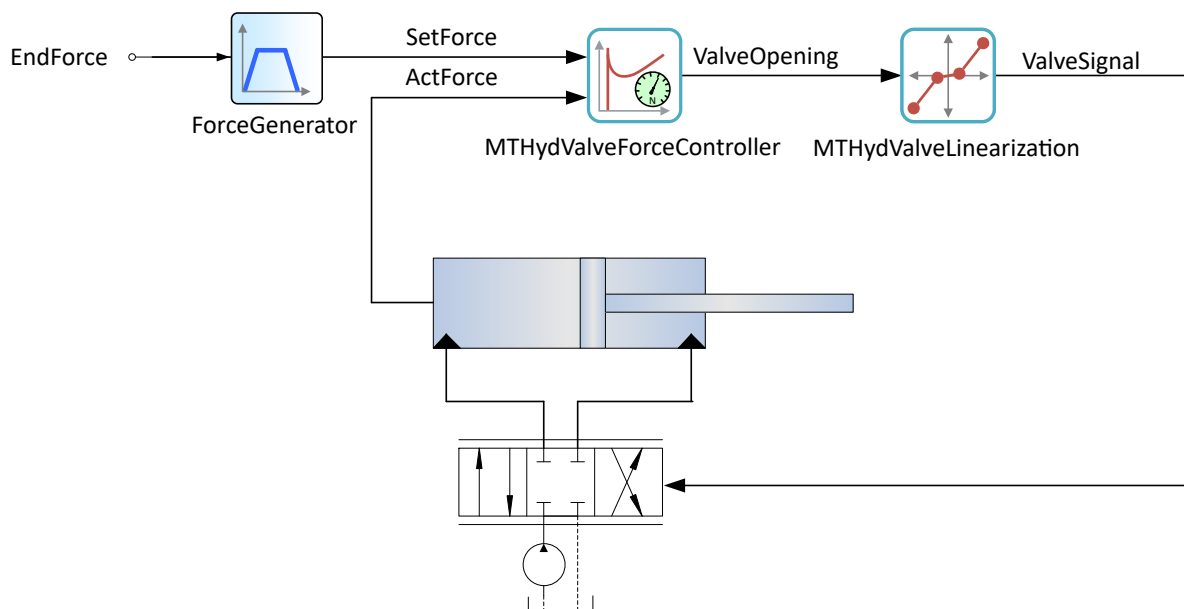


Figure 25: Control loop of a simple force control

7.2 MTHydValveForceController

The MTHydValveForceController function block is a force controller optimized for valve-based hydraulics.

The function block includes two components, a PID controller and dF-y-servo correction. More information about using servo correction can be found in the next section.

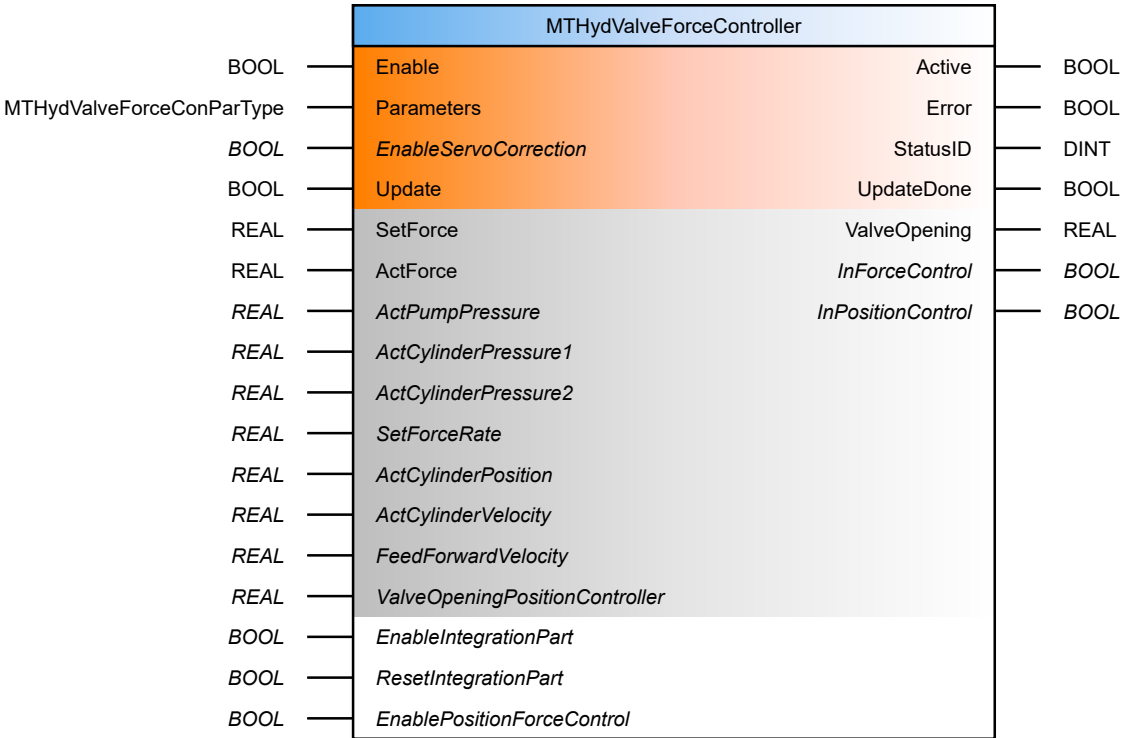


Figure 26: Function block MTHydValveForceController

Programming \ Libraries \ Mechatronic libraries \ Hydraulics \ MTHydValve \ Function blocks \ MTHyd-ValveForceController

In the simplest case, the MTHydValveForceController function block is operated as a pure PID controller (EnableServoCorrection = FALSE). In this mode, only the PID parameters (Parameters.PID) have to be configured. SetForce and ActForce are the only cyclical inputs required for that. The manipulated variable is connected to the ValveOpening output. This variable's default range is [-1..+1].

Changes to the parameter structure and the input EnableServoCorrection do not take effect until the function block is enabled or there is a rising edge of the update input.

If servo correction is additionally used, the cyclic inputs ActPumpPressure, ActCylinderPressure1, ActCylinderPressure2, SetForceRate, ActCylinderPosition, ActCylinderVelocity and the parameter structure Parameters.MachineData must be configured.

7.3 Servo correction

Like the position controller, the force controller also possesses servo correction in order to improve force control. The servo correction of the MTHydValveForceController now calculates a valve opening from the rate of force increase. The rate of force increase setpoint is generated by the profile generator and forwarded to the force controller.

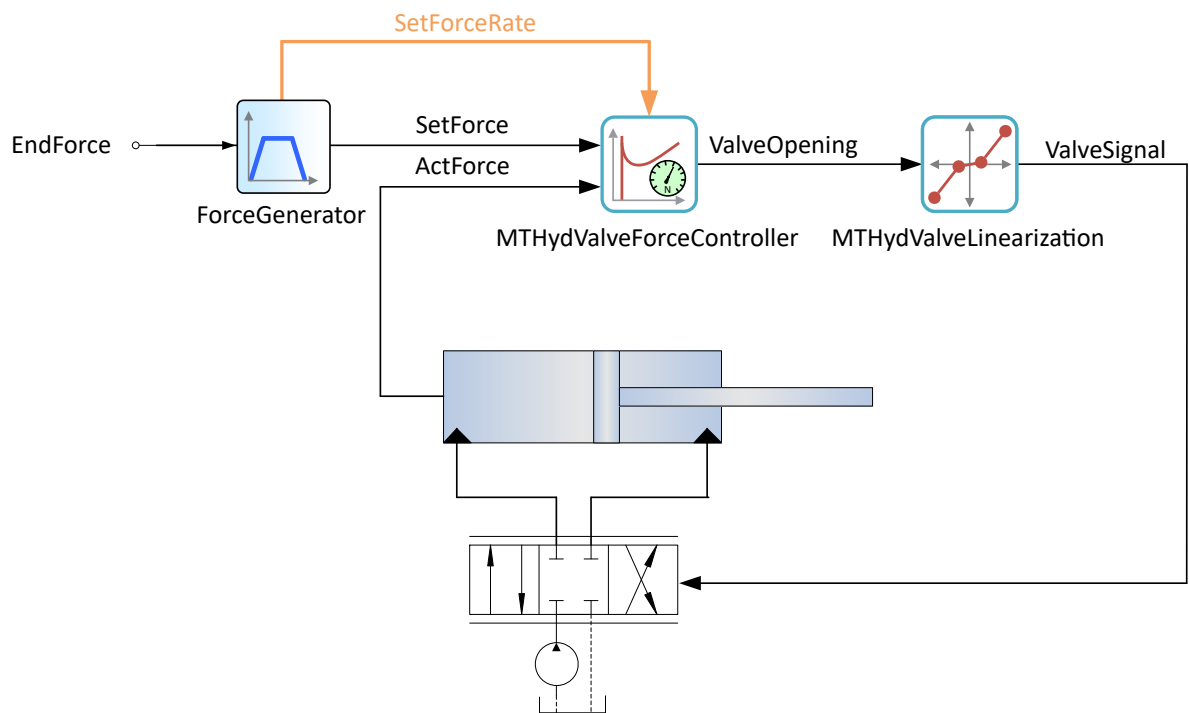


Figure 27: The rate of force increase setpoint as feed-forward

The extended force control block diagram shows that the PID controller now outputs a rate of force increase setpoint. Servo correction dynamically converts this rate of force increase setpoint into a valve opening. Applying valve linearization to the valve opening then results in the valve voltage.

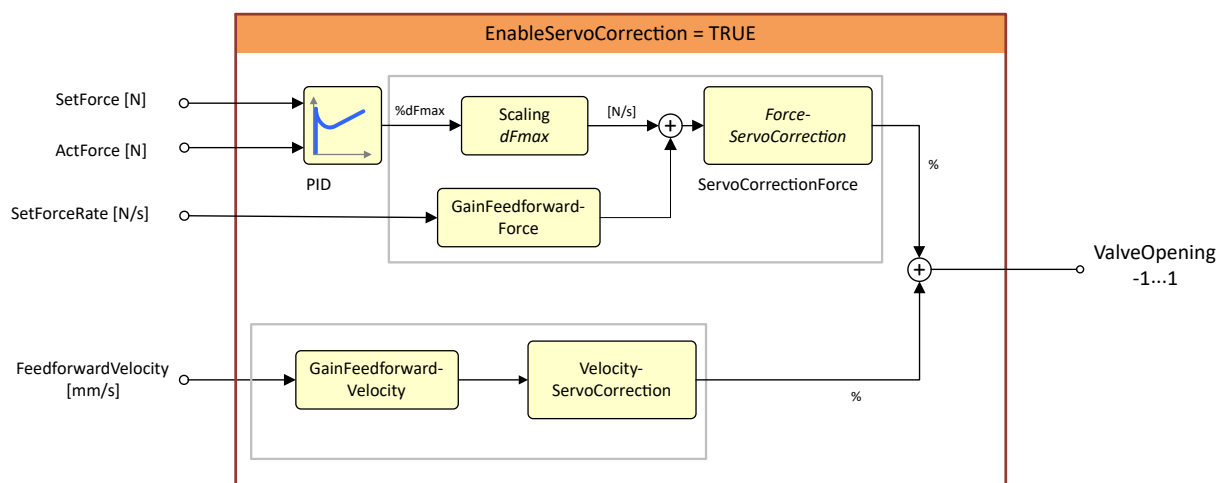


Figure 28: Internal structure of the MTHydValveForceController



In order to use the servo correction of the function block MTHydValvePositionController, the cyclical inputs ActPumpPressure, ActCylinderPressure1, ActCylinderPressure2, SetForceRate, ActCylinderPosition, ActCylinderVelocity and the parameter structure Parameters.MachineData just have to be configured accordingly. Servo correction is enabled with the input EnableServoCorrection.

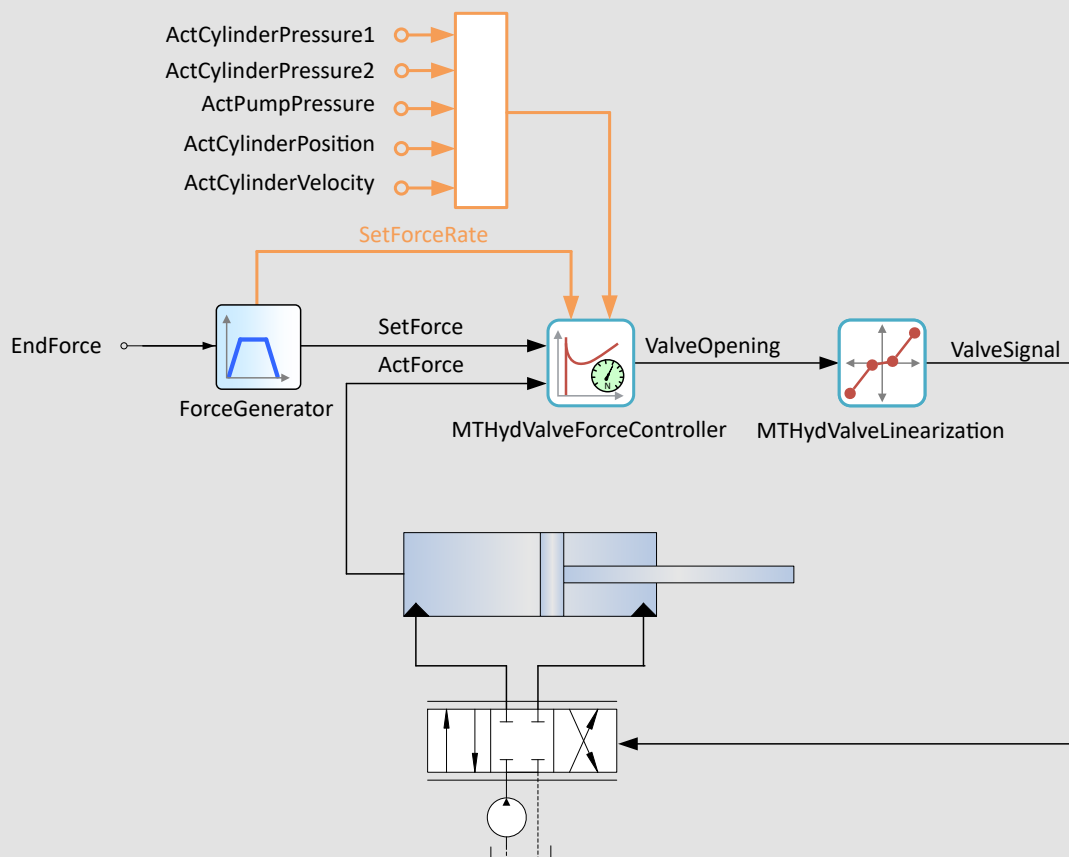


Figure 29: Cyclical inputs for servo correction of the force controller



With input **FeedForwardVelocity**, a cylinder velocity setpoint can be additionally included for feed-forward control of the force control. This way, force control can be executed for a moved cylinder.



As can be seen from the previous image "Internal structure of the MTHydValveForceController", the PID output is scaled to the rate of force increase setpoint when enabling servo correction. This way, the gain of the PID controller has a different effect than when operating without servo correction.

8 Alternating positioning and force control

Many real world applications require a solution that alternates between position and force control. This requirement is characterized by the following cycle:

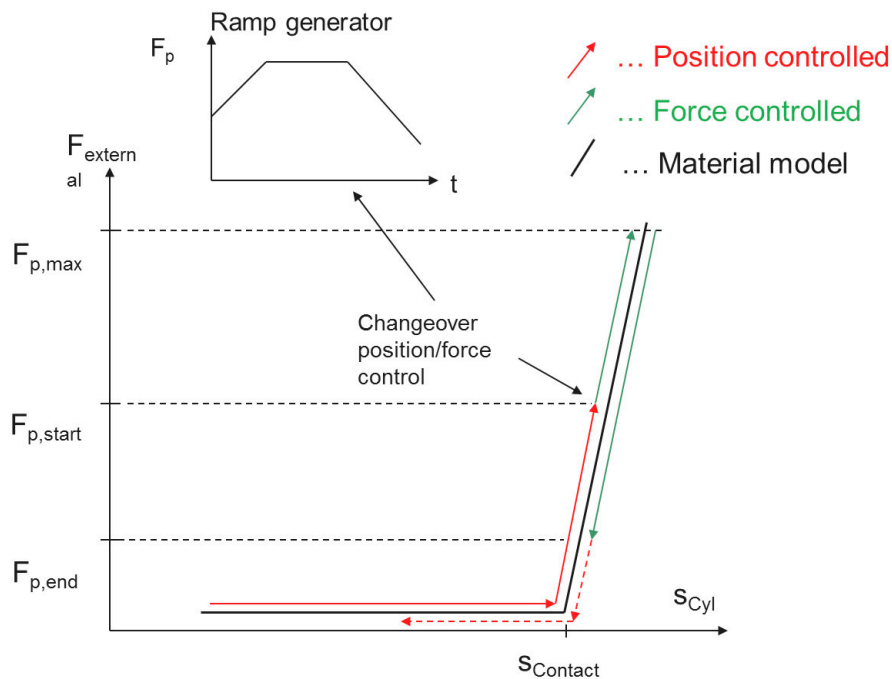


Figure 30: Principle of alternating position and force control

- Position-controlled movement
- Changeover to force control upon reaching an adjustable threshold value for the press force
- Controlled increase of the press force to the target force, maintaining the target force and force reduction until the threshold value is achieved
- Return to initial position

8.1 The principle of alternating control

With alternating control of position and force, the position and force controllers run in parallel. Only one controller is connected to the actuator at a time, however. For this reason, the following two points are crucial:

- Definition of consistent changeover conditions
- Smooth changeover between the controllers

The function block MTHydValveForceController offers the function of alternating control (EnablePositionForceControl = TRUE). Position controllers and force controllers must be operated in parallel. On the input ValveOpeningPositionController, the force controller is informed cyclically of the valve opening of the position controller. Internally, the function block uses a minimal selector. This way, the function block MTHydValveForceController always outputs the valve opening that is smaller on the output ValveOpening. On the outputs InForceControl and InPositionControl, it is possible to read whether the valve opening of the position controller or the valve opening of the force controller is being output. The minimum selector automatically ensures a smooth changeover between position and force control.

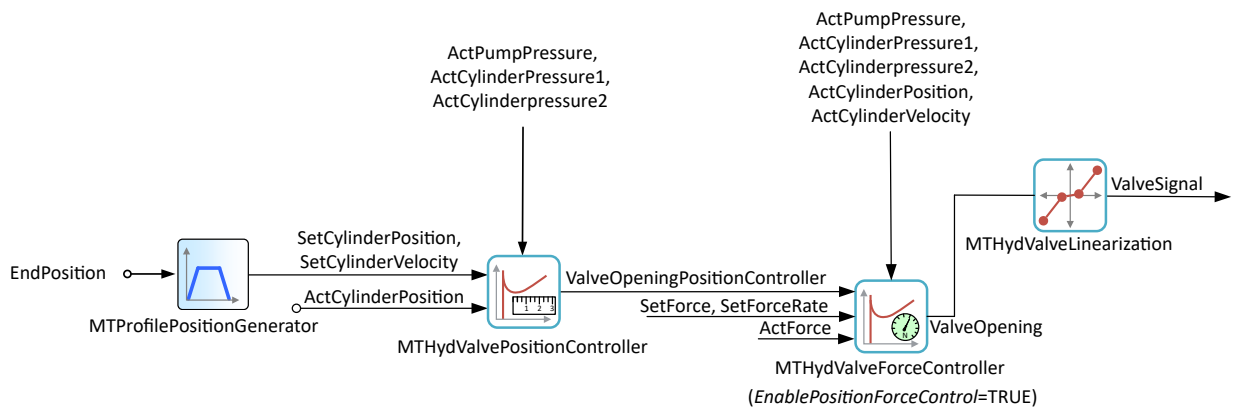


Figure 31: Block diagram of the alternating position and force control using a minimum selector

Alternating control can be used independently of servo correction:

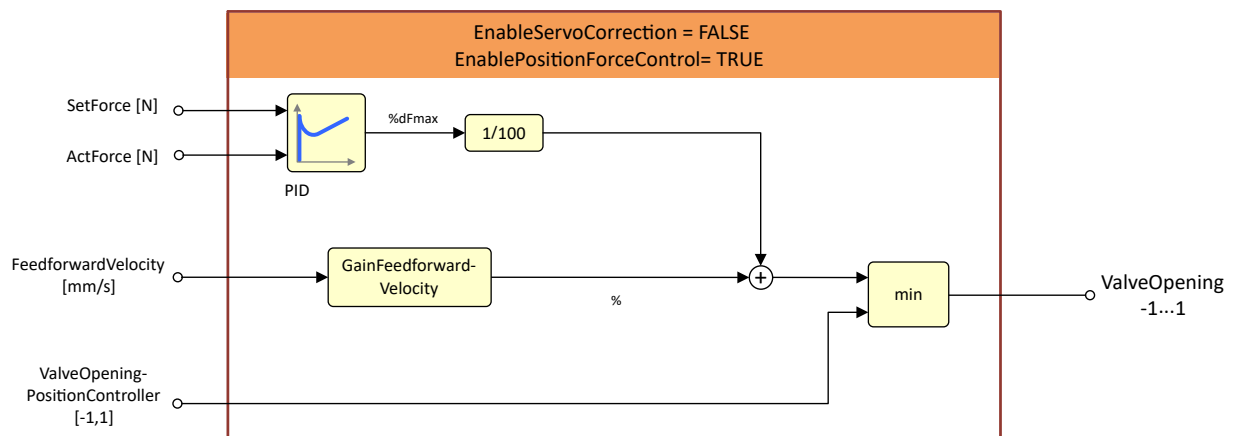


Figure 32: Block diagram of MTHydValveForceController with alternating control and disabled servo correction.

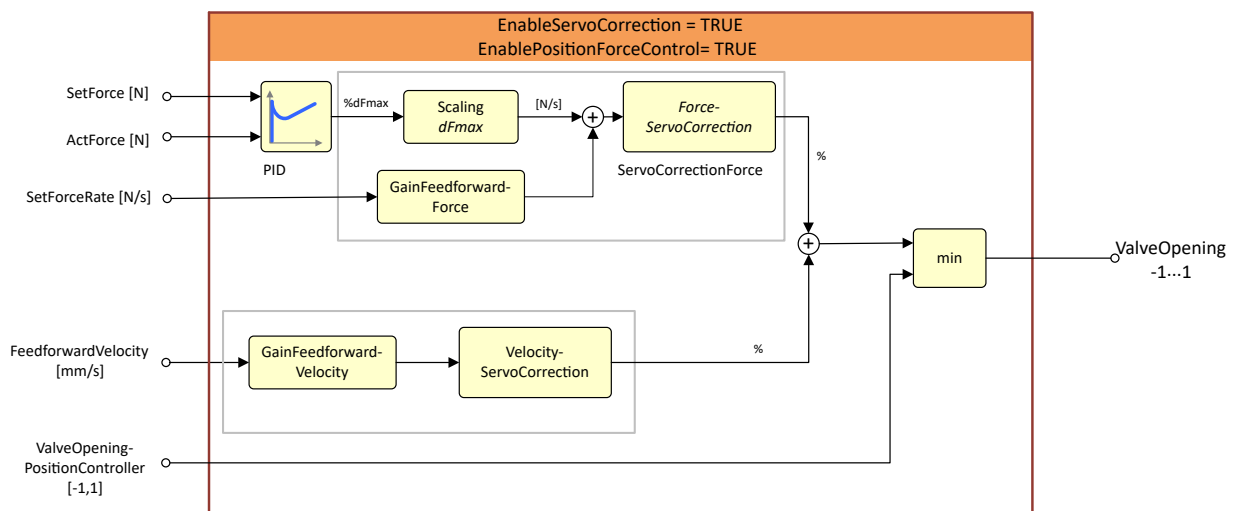


Figure 33: Block diagram of MTHydValveForceController with alternating control and active servo correction.

If the switchover has to take place after a certain condition (e.g. force or position-dependent), force and position controllers must be operated in parallel in the application and the switchover condition programmed manually. Depending on the defined switchover condition (force or position-dependent), either the valve opening of the force controller or position controller must be passed on to the valve linearization. During the switchover, pay attention that no jumps occur on the actuator signal (**ValveOpening**).

8.1.1 Alternating position and force control exercise

Exercise: Carry out alternating position and force control using the Technology Solution

The Technology Solution Hydraulic Valve Control must be put into operation and the HMI application can be used to execute alternating position and force control. On the page Parameters, the parameters for position and force control can be adjusted.

Procedure

- Install the most current version of the Technology Solution "Hydraulic Valve Control" via upgrade.
- Add the Technology Solution in Automation Studio
- Enable the configuration "HydraulicValveControl" added by the Technology Solution.
- Update Runtime version.
- Transfer to target system.
- Connect with VNC viewer.
- Execute position and force control with HMI application.
- Observe in Trend.

9 Project development

9.1 Helpful information regarding project preparation

9.1.1 Hydraulics circuit diagram

The hydraulics circuit diagram should be studied and discussed with the machine operator before the beginning of each hydraulics project.

9.1.2 Machine description

The information for configuring the MTHydValve function block relies significantly on the valves in use, i.e. the hydraulic drive in use, e.g. a linear drive. This information can be found in the respective data sheets.

9.1.3 Hardware selection

Choosing the right hardware depends significantly on the number of hydraulic axes to be operated and the required response time. Recommended values for selecting a response time are provided in the following table.

Medium control quality	Response time: approx. 2 ms
High control quality	Response time: approx. 1 ms
Highest control quality	Response time: approx. 0.5 ms

The following table provides additional recommended values for selecting the appropriate CPU for a predefined response time and number of axes.

PP400 , X20CPx484 or similar	1 – 2 hydraulic axes with a response time of approx. 1.5 ms
X20CPx485 or similar	Up to 4 hydraulic axes with a response time of approx. 1 ms
X20CPx486 or similar	Up to 2 hydraulic axes with a response time of approx. 0.5 ms
	Up to 4 hydraulic axes with a response time of approx. 1 ms
APC620 or similar	Up to 16 hydraulic axes with a response time of approx. 1 ms
	Up to 10 hydraulic axes with a response time of approx. 0.8 ms
	2 hydraulic axes with a response time of approx. 0.4 ms

The response time results from

- inputting the measurement signals from the input modules into the computer via the X2X Link,
- processing the measurement signals and determining the output values by the CPU,
- issuing the output sizes to the output module via the X2X Link.

The response time can be determined using the following formula:

Response time = cycle time of the task class + 2 x X2X Link cycle time + module delay (max. 65 µs)

The Excel sheet provided in the Automation Studio help system can be used to determine the smallest possible X2X Link cycle time as well as the I/O communication time for a given I/O configuration, cable length and X2X settings in Automation Studio.



Communication / X2X / General information / Determining the cycle time

9.1.4 Settings in Automation Studio

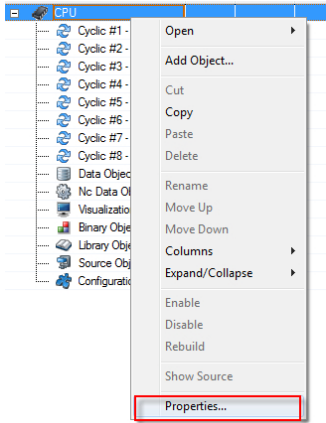
The following guidelines should be observed in order to achieve the minimum response time for the given hardware configuration:

Generally, a hydraulic controller is executed in the fastest task class. The coordination of the task class cycle time with that of the X2X Link is important: the cycle time of the task class must always be a whole-number multiple of the X2X Link cycle time in order to allow synchronous operation.

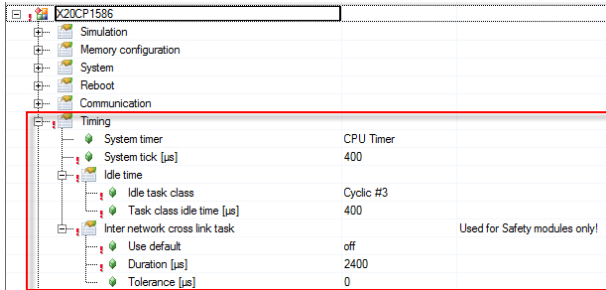
An X2X Link cycle time of 200 µs can be set for most hydraulic control loops. A value of 400 µs is recommended as the cycle time for the fastest task class, if the CPU performance permits it. The stack of the task class must generally be increased compared to the standard setting. In order to avoid memory problems, it is best to start with a value of 65535 bytes. Below you see how to make the settings for the system timer, task class and X2X Link.

System timer settings:

- Open the CPU properties window:

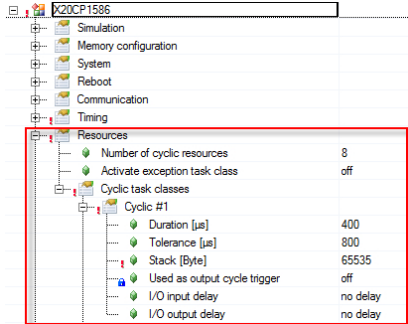


- Go to the "Timing" tab and make the following settings:



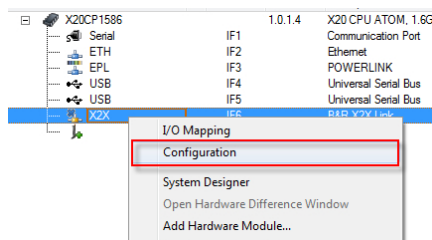
Task class settings:

- In the CPU properties window, go to the "Resources" tab and make the following settings:

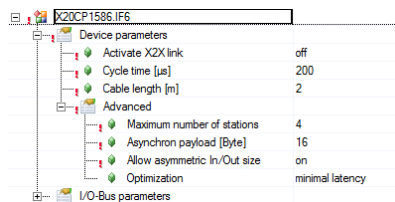


X2X Link settings:

- Open the X2X Link configuration window.



- Make the following settings:



9.2 Commissioning guidelines

In general, the supplier of a hydraulic system must provide commissioning instructions for the system that should be consulted before each commissioning.

The following section covers several additional points to consider with regard to commissioning.

Especially in the case of hydraulic systems, it is important to make detailed preparations in advance and to develop a step-by-step plan that verifies the functionality of each required component before it is commissioned.

Most important, this requires the implementation of appropriate operating modes. In addition to automatic mode, the option of manual operation must always be available as well. With complex systems, it is a good idea to implement a suitable visualization application for manual operation in advance.

The following components should be observed during a function test:



- Only slow movements are performed for testing.
- System pressure is kept as low as possible during commissioning. The purpose of this is to protect all components since the hydraulic oil is generally still contaminated.

- Valves: "Does the valve response (open/closed for switching valves, position of continuous valves) match the control action performed?"
- System pressure pump: "Does the direction of rotation of the electric motor match the intended direction of rotation for the pump?"
- Valve control: The linkage between safety and proportional valves must be taken into account for valve control actions. In general, the safety valves are opened in the first step while the proportional valve is closed. The proportional valves are then controlled in manual mode in order to verify their effective direction (a control block is not used for this; instead, the valve voltage is specified directly).
- During manual operation and at low system pressure, the entire operating area of a hydraulic cylinder, for example, should be run through several times in order to remove potential air pockets before the system pressure is increased.

Gradual implementation of automatic mode should commence only after it has been confirmed that all components are functioning.

9.2.1 Controller configuration

MTHydValve library control blocks are generally configured in two steps:

- Using valve parameters, valve characteristic curves, cylinder dimensions, etc. that are listed in data sheets
- By configuring a higher-level PI controller

The first point has already been described in detail in this training module. Training module "TM 260 – The Basics of Closed-loop Control" contains much useful information regarding the second point. The following is a possible approach for configuring the control blocks provided in the MTHydValve library.

The procedure for tuning the hydraulic controller is described here using position control with servo correction.

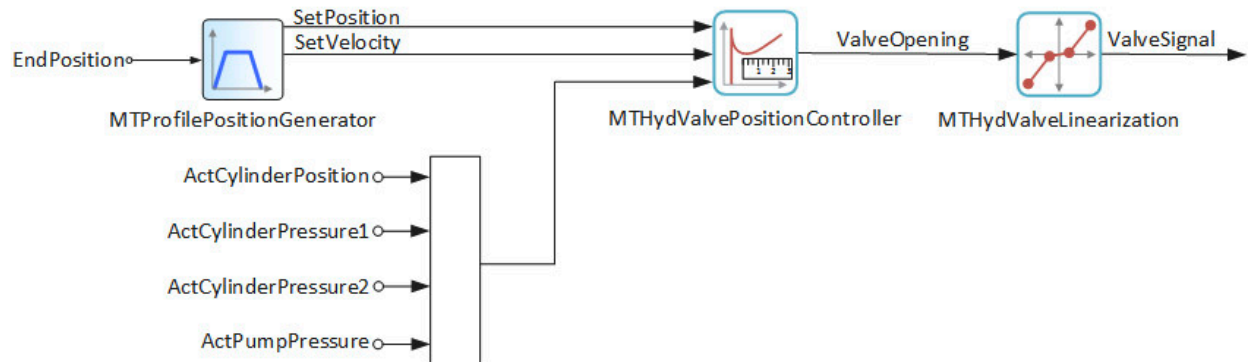


Figure 34: Closed-loop control structure for position control with servo correction and pressure sensors.

Procedure for tuning the hydraulic controller

- 1) The first step is to operate the control loop with servo correction only (feed-forward control).
 - `SetCylinderVelocity = MTProfilePositionGenerator.Velocity`
 - `Gain = 0.0`
 - `IntegrationTimePositive = 0.0`
 - `IntegrationTimeNegative = 0.0`
 - `DerivativeTime = 0.0`
 - `FilterTime = 0.0`
- 2) To display the current cylinder velocity, the current cylinder position can be derived using function block `MTBassicsDT1`.
- 3) Compare the velocity setpoint with the actual velocity of the cylinder after step 1 has been completed. Ideally – i.e. if all machine data was specified correctly and friction/leakage have only a minimal influence – the deviation will be relatively small.
- 4) If the deviation between the actual velocity and velocity setpoint is nevertheless too high, input `GainSetVelocity` can be used to implement a gain factor in order to influence the behavior of servo correction.
 - Actual velocity < Velocity setpoint: `GainSetVelocity > 1`
 - Actual velocity > Velocity setpoint: `GainSetVelocity < 1`

If deviations are much too high, however, check whether the machine data was entered correctly or whether other factors (friction, too much leakage) are responsible.

- 5) If the cylinder is insufficiently positioned with servo correction, the P controller can be used. Its only task is to correct the remaining deviation between the actual position and position setpoint. To do so, increase parameter `PIDParameters.Gain` in small steps starting from 0.0. By moving the cylinder and observing the control behavior, it should be possible to find the optimal gain factor. Increasing this value makes the control loop faster but also increases the tendency to oscillate. It is particularly important to test the controller with the piston in the center position since this is where the tendency to oscillate is highest.
- 6) It should be possible to compensate for a deviation that remains after servo correction and the P controller using the controller's I component (`PIDParameters.IntegrationTimePositive`, `PIDParameters.IntegrationTimeNegative`). The integral component should only be enabled at very low velocities and shortly before reaching the target position; otherwise, it should remain disabled off to prevent overshoot. Function block `MTHydValvePositionController` provides input `EnableIntegrationPart` for this. The general rule is, the smaller the integration time constant, the more aggressive the integral component responds.
- 7) It is important to note that I components at standstill with high static friction in the system can lead to "stick-slip" effects. Stick-slip effects result if the static friction is greater than the kinetic friction. In these cases, a strong force is needed to overcome the static friction. Once the static friction has been overcome, this force results in a movement past the position setpoint. The same effect repeats itself in the opposite direction. What can result is a persistent oscillation around the position setpoint. The way to prevent this is to set up a dead zone for the I component. As long as the value of the control deviation is absolutely less than a corresponding value, the I component is simply frozen, which prevents the stick-slip effect.



If servo correction is enabled or disabled after the PID controller is tuned, then the proportional gain of the PID controller must generally be changed as well.

Automation Academy

Your knowledge advantage

The Automation Academy provides **targeted training** courses for our customers as well as for our own employees. Expand your skills in the field of automation technology and learn to independently implement **efficient automation solutions** using B&R systems.

Decide for yourself which **learning concept** you prefer!



Classroom Learning



Virtual Classroom
Learning



Online courses

Classroom learning

B&R offers **standard seminars** at all B&R locations. Services include seminar documents, effective communication of training course content by experienced trainers and an Automation Diploma. A combination of group work and self-study provides the high level of flexibility needed to maximize the learning experience.

Virtual classroom

Remote Lectures supplement B&R's continuing education portfolio with a virtual classroom, offering an alternative to our on-site seminars. Selected content from our standard seminars is offered online. In addition to remote learning methods, powerful simulation tools and secure remote maintenance are used.

Online courses

Take control of the content and learn at your own pace. With B&R **online courses**, you can take your first steps in the world of B&R automation at any time. Based on a comprehensive narrative, you will independently work out how to use our products. The mix of different media allows a logical sequence to be followed when learning as well as a targeted choice of information to be used as a reference.

Contact

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place.

Access additional information here:

<https://www.br-automation.com/de/academy/>

Enjoy your next training course!



AutomationAcademy





B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

